

Apellidos:

Nombre:

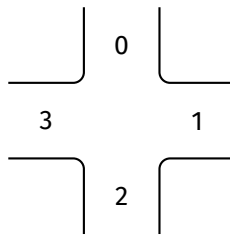
Concurrencia y Paralelismo

Grado en Ingeniería Informática

Mayo 2019

1. Control de tráfico en una intersección [2 puntos]

Vamos a simular una intersección en una carretera donde hay un semáforo que deja pasar a los coches que llegan desde cada una de las direcciones consecutivamente.



Tanto el semáforo como cada coche que llega a la intersección tiene un thread propio. La intersección se representa con un valor de tipo `intersection`, donde hay un campo `direction` que indica la dirección que puede cruzar ahora mismo, y una constante `total_directions` que indica el número total de direcciones en la intersección.

El código siguiente implementa el cambio periódico de dirección:

```
typedef struct _intersection {
    int direction;           // Current direction
    int total_directions;    // Total number of directions
    pthread_mutex_lock *i_lock;
    ...
} *intersection;

int semaphore(intersection i) {
    while(1) {
        pthread_mutex_lock(i->i_lock);
        i->direction = (i->direction + 1) % i->total_directions;
        pthread_mutex_unlock(i->i_lock);

        sleep(10);
    }
}

int car(int direction, intersection i) {
    ...
}
```

Implemente el comportamiento del coche para que espere hasta que el semáforo deje pasar a los coches de su dirección. Puede añadir campos a la estructura `intersection` y cambiar la implementación de la función `semaphore` si es necesario.

```
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Solución

```
typedef struct _intersection {
    int direction;           // Current direction
    int total_directions;    // Total number of directions
    pthread_mutex_t *i_lock;
    pthread_cond_t *i_wait; // malloc(sizeof(pthread_cond_t) * total_directions);
                           // (equivalent to pthread_cond_t i_wait[total_directions])
                           // initialized using pthread_cond_init(i_wait[i], NULL)
                           // in a loop
} *intersection;

int semaphore(intersection i) {
    while(1) {
        pthread_mutex_lock(i->i_lock);
        i->direction = (i->direction + 1) % i->total_directions;
        pthread_cond_broadcast(i->i_wait[i->direction]); // Wake up the cars waiting for
                                                         // green after changing
        pthread_mutex_unlock(i->i_lock);

        sleep(10);
    }
}

int car(int direction, intersection i) {
    pthread_mutex_lock(i->i_lock);
    while(direction != i->direction)                    // Check if the semaphore is
                                                         // green for us while holding
                                                         // i->lock

        pthread_cond_wait(i->i_wait[direction], i->i_lock); // If it isn't, wait.
                                                         // Release i_lock while waiting,
                                                         // which will let the semaphore
                                                         // change i->direction

    pthread_mutex_unlock(i->i_lock);
}
```

2. Comunicación entre barberos y clientes [1.75 puntos]

Dado el algoritmo de barberos dado en clase expuesto a continuación, conseguir que el barbero sepa a que cliente le está cortando el pelo y viceversa para que puedan imprimirse los mensajes con los valores correctos. Téngase en cuenta que debe funcionar tanto si el cliente espera en la sala de espera como si no hay espera. Y que el algoritmo tiene que funcionar cuando hay más de un barbero.

```
void barber(int barber)
{
    int customer = -1;
    while(1) {
        pthread_mutex_lock(&mutex);
        if (!waiting_customers) {
            free_barbers++;
            pthread_cond_wait(&no_customers, &mutex);
        }
        pthread_cond_signal(&waiting_room);
        waiting_customers--;
    }
    pthread_mutex_unlock(&mutex);
    printf("barber %d cut hair of customer %d\n", barber, customer);
}

void customer(int customer)
{
    int barber = -1;
    pthread_mutex_lock(&mutex);
    if(waiting_customers == MAX_CUSTOMERS)
        pthread_mutex_unlock(&mutex);
    else {
        if(barberos_libres>0) {
            pthread_cond_signal(&no_customers);
            free_barbers--;
        }
        waiting_customers++;
        pthread_cond_wait(&waiting_room, &mutex);

        pthread_mutex_unlock(&mutex);
        printf("customer %d got hair cut from barber %d\n", customer, barber);
    }
}
```

Puede asumir que dispone de la siguiente estructura de datos:

```
typedef struct _queue *queue;    // a FIFO structure

void insert (queue q, void *e); // Inserts e into q
void *remove (queue q);         // removes an element from q
int  elements(queue q);         // returns the number of elements in q
```

Solución

```
struct c_info {
    int customer;
    int barber;
    pthread_cond_t wait;
};

queue q;

void barber(int barber)
{
    int customer = -1;
    struct c_info *element;

    while(1) {
        pthread_mutex_lock(&mutex);

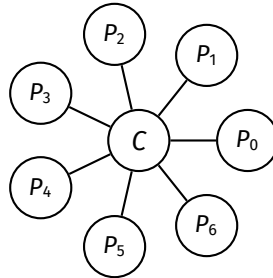
        if (!waiting_customers) {
            free_barbers++;
            pthread_cond_wait(&no_customers, &mutex);
        }
        element = remove(q);
        q->barber = barber;
        customer = q->customer;
        pthread_cond_signal(&element->wait);
        waiting_customers--;
        pthread_mutex_unlock(&mutex);
        printf("barber %d cut hair of customer %d\n", barber, customer);
    }
}

void customer(int customer)
{
    int barber = -1;
    struct c_info element;

    pthread_mutex_lock(&mutex);
    if(waiting_customers == MAX_CUSTOMERS)
        pthread_mutex_unlock(&mutex);
    else {
        if(free_barbers > 0) {
            pthread_cond_signal(&no_customers);
            free_barbers--;
        }
        waiting_customers++;
        element.customer = customer;
        insert(q, &element);
        pthread_cond_wait(&element.wait, &mutex);
        pthread_mutex_unlock(&mutex);
        printf("barber %d cut hair of customer %d\n", element.barber, customer);
    }
}
```

3. Comunicación en estrella [1.25 puntos]

En un sistema de comunicación en estrella hay un proceso central que se encarga de propagar los mensajes de los nodos situados en el exterior de la estrella.



El nodo C recibe mensajes de los nodos P_i y los copia al resto de nodos de la estrella (es decir, a todos menos al que originó el mensaje). Por ejemplo, si en diagrama de ejemplo el nodo P_0 envía un mensaje debe ser recibido por $P_1, \dots, P_6$.

El siguiente módulo es un esqueleto del código para el nodo central C , con dos funciones interfaz:

- a) `start(Pids)`, donde `Pids` es una lista con los Pids de los procesos que forman parte de la estrella (en el ejemplo `[P_0, \dots, P_6]`)
- b) `send(Center, Msg)`, donde `Center` es el Pid de C , y `Msg` es el mensaje que se desea enviar a los otros procesos de la estrella. Esta función será llamada por los P_i .

```
-module(star).  
  
-export([start/1, send/2]).  
  
start(Pids) ->  
    spawn(?MODULE, loop, [Pids]).  
  
send(Center, Msg) ->  
    ...  
  
loop(Pids) ->  
    receive  
        ...  
    end.
```

Implemente las funciones `loop` y `send` para que los mensajes enviados por los procesos a través de `send` lleguen a todos los demás procesos de la estrella.

Solución

```
-module(star).

-export([start/1, send/2]).

start(Pids) ->
    spawn(?MODULE, loop, [Pids]).

send(Center, Msg) ->
    Center ! {send, Msg, self()}.          %% Send the message to C. C has to
                                           %% filter the sender process,
                                           %% so we include self() in the message

loop(Pids) ->
    receive
        {send, Msg, From} ->              %% C receives a message Msg from From, so
                                           %% it has to resend it to everyone in Pids
                                           %% excluding From

            send_to_all(Pids, Msg, From),  %% We do that in send_to_all
            loop(Pids)                    %% and loop to process further requests

    end.

send_to_all([], _, _) ->                  %% Empty List, no more processes to send to
    ok;

send_to_all([From | T], Msg, From) ->    %% Head of the List is the process that sent Msg
    send_to_all(T, Msg, From);           %% send to the rest of the processes

send_to_all([P | T], Msg, From) ->      %% P is not equal to From,
    P ! Msg,                             %% so send Msg to it,
    send_to_all(T, Msg, From).           %% and send to the rest of the processes
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

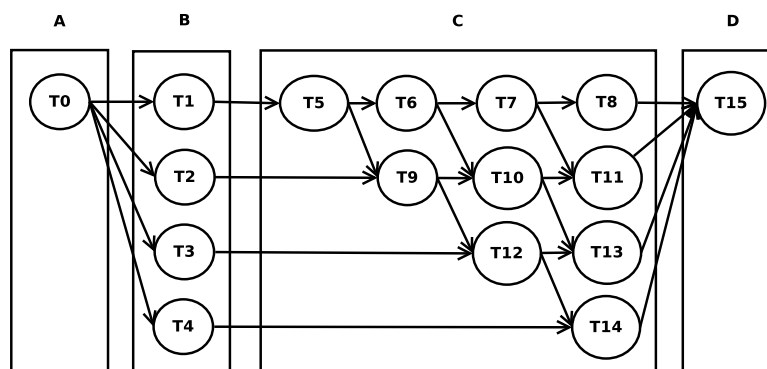
Convocatoria ordinaria

21 de mayo de 2019

AVISO: No mezcleis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregan por separado.

[2,5p] 1. **Conceptos de paralelismo.** Considera un programa con cuatro regiones de código (A, B, C y D) ejecutadas una tras otra en su versión secuencial inicial. Las regiones A y D no se pueden paralelizar, necesitando 200 y 300 operaciones, respectivamente. Por el contrario, las regiones B y C sí pueden ser paralelizadas, realizando 16.000 y 25.000 operaciones, respectivamente.

- (a) Calcula, según la ley de Amdhal, cuál es la máxima aceleración que puede obtener este programa. [0,5p].
- (b) Tras aplicar la descomposición de tareas a las regiones B y C, nos damos cuenta de que la región B puede ser ejecutada completamente en paralelo mientras que la región C sí presenta dependencias entre algunas de sus tareas. La siguiente figura muestra el grafo de dependencias. Sabiendo que las tareas de la región B necesitan 4.000 operaciones cada una, mientras que las de la región C necesitan 2.500 operaciones cada una, ¿cuál es el grado medio de concurrencia? [0,75p]



- (c) Si las tareas son asignadas a cuatro procesos como sigue: $P0=\{T0, T1, T5, T6, T7, T8, T15\}$, $P1=\{T2, T9, T10, T11\}$, $P2=\{T3, T12, T13\}$ y $P3=\{T4, T14\}$ ¿cuál sería el speedup que podría obtenerse? ¿y la eficiencia paralela? [0,5p]
- (d) De cara a mejorar el speedup obtenido con cuatro procesos, se propone dividir de nuevo CADA UNA de las tareas de la región B en cuatro subtareas completamente independientes. Por ejemplo, T1 sería reemplazada por T1.1, T1.2, T1.3 y T1.4, cada una con 1.000 operaciones y con T5 dependiendo de todas ellas (de forma análoga para T2, T3 y T4, con T9, T12 y T14 dependiendo de ellas, respectivamente). ¿Se mejora el grado medio de concurrencia con esta aproximación? Proporciona una asignación de tareas que obtenga el mejor speedup posible para cuatro procesos e indica el valor de ese speedup. [0,75p]

SOLUCIÓN

- (a) El problema tiene una parte intrínsecamente secuencial con 500 operaciones y otra perfectamente paralelizable con 41.000 operaciones. Aplicando la Ley de Amdahl:

$$sp(p \rightarrow \infty) = \frac{T_{sec}}{T_{par}(p \rightarrow \infty)} = \lim_{p \rightarrow \infty} \frac{500 + 41.000}{500 + \frac{41.000}{p}} = \frac{41.500}{500} = 83$$

- (b) El camino crítico comprende a T0 (200 operaciones), una de las sub tareas de la región B (4.000 operaciones), cuatro de las sub tareas de la región C (10.000 operaciones) y T15 (300 operaciones). Por tanto el camino crítico necesita 14.500 operaciones. El grado medio de concurrencia es el número total de operaciones entre la longitud del camino crítico:

$$\frac{41.500}{14.500} \simeq 2.86$$

- (c) El siguiente gráfico muestra la distribución de tareas entre los procesos.

P0	T0		T1		T5		T6		T7		T8		T15
P1			T2				T9		T10		T11		
P2			T3						T12		T13		
P3			T4								T14		

El tiempo paralelo es el tiempo del proceso con más operaciones (P0): 14.500 operaciones. Por tanto:

$$Sp(4) = \frac{41.500}{14.500} \simeq 2.86$$

$$Eff(4) \frac{2.86}{4} \simeq 0.72$$

- (d) Sí se mejora el grado medio de concurrencia porque ahora no toda la T1 está en el camino crítico sino solo un cuarto de ella. Por tanto este camino crítico tiene una longitud de 11.500 operaciones en lugar de 14.500. Esto indica un grado medio de concurrencia de:

$$\frac{41.500}{11.500} \simeq 3.61$$

A continuación se muestra una asignación que logra un speedup máximo (igual al grado medio de concurrencia):

- P0 = T0, T1.1, T5, T6, T7, T8, T15
- P1 = T1.2, T2.1, T2.4, T9, T10, T11
- P2 = T1.3, T2.2, T3.1, T3.3, T4.1, T12, T13
- P3 = T1.4, T2.3, T3.2, T3.4, T4.2, T4.3, T4.4, T14

P0	0	1.1		5		6		7		8		15
P1		1.2	2.1	2.4		9		10		11		
P2		1.3	2.2	3.1	3.3	4.1		12		13		
P3		1.4	2.3	3.2	3.4	4.2	4.3	4.4		14		

[2,5p] 2. **Diseño de algoritmos paralelos.** Queremos paralelizar el siguiente código que trabaja con matrices 2D siguiendo el llamado algoritmo *Rojo-Negro*.


```

void main(int argc, char *argv[]) {
    double tmp;
    double u[N][N];
    LeeMatriz(u);
    // Bucle rojo: atravesando todas las celdas rojas
    for(int i=2; i<N-1; i+=2)
        for(int j=1; j<N-1; j++){
            tmp = u[i+1][j]+u[i-1][j]+u[i][j+1]+u[i][j-1]-4*u[i][j];
            u[i][j] = tmp/4;
        }

    // Bucle negro: atravesando todas las celdas negras
    for(int i=1; i<N-1; i+=2)
        for(int j=1; j<N-1; j++){
            tmp = u[i+1][j]+u[i-1][j]+u[i][j+1]+u[i][j-1]-4*u[i][j];
            u[i][j] = tmp/4;
        }

    EscribeMatriz(u);
}

```

- (a) Dibuja una matriz de dimensiones 9x9 que represente a u en el caso de que N es igual a 9. Indica para cada celda si es roja (R), negra (N) o blanca (B, aquellas celdas que no se calculan). [0,5p]
- (b) ¿Existen dependencias dentro del bucle rojo? ¿Y dentro del negro? ¿Y entre los dos bucles? Razona tus respuestas. [0,5p]
- (c) ¿Qué tipo de descomposición de tareas debemos hacer? ¿Qué tipo de asignación de tareas? [0,5p]
- (d) Teniendo en cuenta tus respuestas a los apartados anteriores, para una ejecución con tres procesos y N igual a 9 indica a qué elementos debe acceder cada uno de los procesos. Puedes usar una representación gráfica. [0,5p]
- (e) Para una ejecución como la del apartado anterior, indica qué elementos deben comunicarse entre que se termina el bucle rojo y comienza el negro. Para cada mensaje debes indicar las filas/columnas enviadas, el tamaño del mensaje, el proceso emisor, y el receptor. [0,5p]

SOLUCIÓN

B	B	B	B	B	B	B	B	B
B	N	N	N	N	N	N	N	B
B	R	R	R	R	R	R	R	B
B	N	N	N	N	N	N	N	B
B	R	R	R	R	R	R	R	B
B	N	N	N	N	N	N	N	B
B	R	R	R	R	R	R	R	B
B	N	N	N	N	N	N	N	B
B	B	B	B	B	B	B	B	B

- (a)
- (b) Las dependencias dentro de un mismo bucle son por columnas, ya que para calcular el elemento (i, j) se necesita el valor del elemento $(i, j-1)$ que ha sido calculado antes. Las dependencias de filas a causa del elemento $(i-1, j)$ no se dan dentro del mismo bucle (ya que cada uno se salta una fila) sino entre los dos bucles.
- (c) Se realizan los mismos cálculos sobre todos los elementos negros y rojos, por tanto una descomposición de dominio sobre la matriz u es adecuada para este algoritmo. Se podría llevar a cabo tanto por filas como por columnas, elegimos la primera porque suele ser más fácil de implementar posteriormente con lenguajes basados en C (con los elementos almacenados en memoria de forma consecutiva por filas). Cada fila sería una tarea.
- A la hora de asignar, como existen dependencias de filas entre los bucles, lo más lógico es hacer una asignación bloque donde filas consecutivas pertenezcan a un mismo proceso. Esto puede desbalancear un poco ya que la primera y última fila (asignadas al primer y último proceso, respectivamente) no realizan cálculo. Pero este desbalanceo no debería ser importante en tamaños grandes de matrices. Para poder hacer todos los cálculos los procesos necesitan las filas inmediatamente superiores e inferiores a las que tienen que calcular.
- (d) Cada uno de los procesos calcula tres filas de forma bloque. El primer proceso tendría que acceder a las cuatro primeras filas (la cuarta es necesaria para poder calcular la tercera). De forma análoga el último proceso debe acceder a las cuatro últimas filas. El proceso del medio debe acceder a las filas 2, 3, 4, 5 y 6, para poder calcular las filas 3, 4 y 5.
- (e) Como el bucle rojo es anterior al bucle negro solo hay que fijarse en las dependencias rojo-negro. En este caso hay dos que involucran mensajes. Por un lado P1 necesita tener actualizada la fila 2 (calculada por P0). Eso se traduce en que P0 manda a P1 los siete elementos centrales de la fila 2. También P1 necesita tener actualizada la fila 6 para poder calcular la fila 5. Eso se traduce en que P2 manda a P1 los siete elementos centrales de la fila